

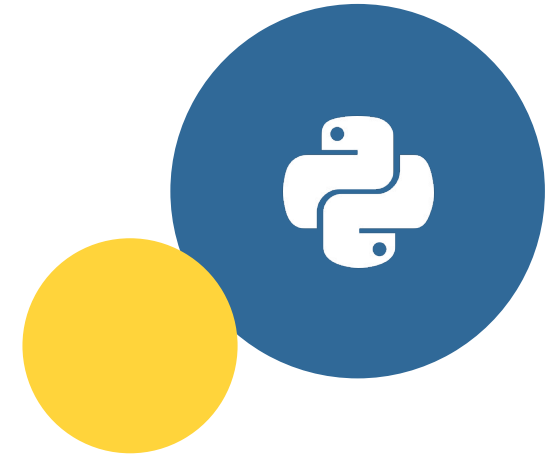
MÓDULO 2

Operadores, Expressões e Variáveis

Introdução à Programação Estruturada em Python

Ciência da Computação · Sistemas de Informação — 2º Semestre

Professores: Luciano · Rafael · Turatti



```
aula.py

nota = 8.5
media = (nota + 7.0) / 2
print(media) # 7.75
```

variáveis

constantes

expressões

operadores

atribuição

precedência

Roteiro do módulo

1



Conceito de variável

O que é, como vive na memória e regras de nomes

2



Constantes e números

int, float e a convenção de constantes

3



Expressões

Como o Python avalia e produz valores

4



Operadores matemáticos

+, -, *, /, //, % e ** na prática

5



Operadores de atribuição

= e a família composta: +=, -=, *= ...

6



Precedência dos operadores

A ordem que decide o resultado

O que é uma variável?



Uma variável é um nome que referencia um valor guardado na memória do computador.



Criada na atribuição

Python não exige declaração prévia: o nome passa a existir quando recebe um valor.



Dinamicamente tipada

O tipo pertence ao valor, não ao nome — `type()` confirma na hora.



Reatribuível — por isso, variável

O mesmo nome pode receber novos valores quantas vezes quiser.

etiqueta (nome)

idade



20

conteúdo (valor)

o nome aponta para o valor — a caixa mora na memória



variavel.py

```
idade = 20
idade = 21 # o 20 é descartado
print(idade) # 21
```

Atribuição e tipos

tipos.py

```
nome = "Ana"      # str → texto
idade = 20        # int → inteiro
altura = 1.68     # float → real
aprovado = True   # bool → lógico
print(type(idade)) # <class 'int'>
```

python3

```
>>> type(nome) → <class 'str'>
>>> type(altura) → <class 'float'>
>>> type(aprovado) → <class 'bool'>
```

COMO LER

x = 5

lê-se: “x recebe 5” — a direita é avaliada primeiro e o resultado é guardado no nome da esquerda.

- A direita é avaliada primeiro; o resultado é guardado no nome da esquerda.
- O tipo vem do valor: "Ana" gera str, 20 gera int.
- type() devolve o tipo de qualquer valor ou variável.

OS 4 TIPOS DO MÓDULO

str

"Ana"

texto

int

20

inteiro

float

1.68

real (decimal)

bool

True / False

lógico

Regras para nomes



Nomes válidos



salario_bruto



nota1



_total



mediaFinal



Nomes inválidos

~~2nota~~

— não pode começar com dígito

~~nome-aluno~~

— hífen não é permitido

~~salario bruto~~

— espaço não é permitido

~~class~~

— palavra reservada da linguagem



BOAS PRÁTICAS (PEP 8)

Prefira snake_case minúsculo — media_das_notas. **Nomes diferenciam maiúsculas: Nota ≠ nota ≠ NOTA.**

Regra geral: letras, dígitos e _ ; nunca começar por dígito. Acentos funcionam no Python 3, mas evite por compatibilidade.

Constantes



Convenção, não travamento

Python não bloqueia o valor: o nome em MAIÚSCULAS sinaliza “não altere este valor”.



Contrato de leitura

Quem lê o código entende que aquele valor é fixo em todo o programa.



Onde brilham

Fatores de conversão, taxas, limites e mensagens padrão do sistema.



Tecnicamente $\pi = 4$ rodaria sem erro — **a convenção existe para o leitor do código.**



constantes.py

```
PI = 3.14159
TAXA_IPI = 0.25
LIMITE_ALUNOS = 60

area = PI * 10 ** 2    # 314.159
```



python3

```
>>> area    → 314.159
>>> LIMITE_ALUNOS    → 60
```

Tipos numéricos

int

Inteiros

`10 · -3 · 0 · 1_000_000`

Precisão ilimitada: cresce conforme a necessidade.

float

Reais (ponto flutuante)

`1.68 · -0.5 · 2e3 · 3.14`

Aproximação binária: sujeito a pequenos arredondamentos.

complex

Números complexos

`2 + 3j`

Parte real + imaginária; presente em cálculos científicos.

```
>>> type(42) → <class 'int'>
>>> type(3.14) → <class 'float'>
>>> 2e3 → 2000.0
>>> 1_000_000 → 1000000
```

python3



Curiosidade: `0.1 + 0.2 == 0.3` devolve False — arredondamento normal do float: `0.1 + 0.2 → 0.30000000000000004`

O que são expressões?



Expressão é qualquer trecho de código que, ao ser avaliado, **produz um valor**.

ANATOMIA



ARITMÉTICA

```
(2 + 3) * 4  
→ 20
```

TEXTO (CONCATENAÇÃO)

```
"Intro" + "Python"  
→ "IntroPython"
```

COM VARIÁVEIS

```
area = PI * raio ** 2  
o resultado é guardado em area
```

Expressões vivem no lado direito do =, dentro de `print()` e nas condições de `if`.

Operadores matemáticos

Operador	Nome	Exemplo	Resultado
+	Adição	7 + 2	9
-	Subtração	7 - 2	5
*	Multiplicação	7 * 2	14
/	Divisão real	7 / 2	3.5
//	Divisão inteira	7 // 2	3
%	Resto da divisão	7 % 2	1
**	Potência	7 ** 2	49



Atenção: / sempre devolve float — até 8 / 2 → 4.0

Todos com a mesma base: 7 e 2. Os próximos slides detalham os que mais geram dúvida: /, //, % e **.

Divisão real vs. inteira



Divisão real

sempre devolve float

```
>>> 7 / 2 → 3.5  
>>> 8 / 2 → 4.0 # sempre float!
```



Divisão inteira

descarta as casas decimais

```
>>> 7 // 2 → 3 # descarta as casas  
>>> 7.0 // 2 → 3.0 # float entra, float sai  
>>> -7 // 2 → -4 # arredonda para BAIXO
```



-7 // 2 resulta em -4, não em -3: // arredonda para o piso — o menor inteiro possível.



Regra de bolso: // responde “quantas caixas cheias?” • % responde “quanto sobra?”

Módulo: o operador %

```
python3 - % (resto)

>>> 7 % 2 → 1
>>> 10 % 3 → 1
>>> 12 % 4 → 0 # divisão exata
```

A IDENTIDADE POR TRÁS

$7 = 2 \times 3 + 1 \rightarrow (7 // 2, 7 \% 2) = (3, 1)$



Analogia do relógio: $13 \% 12 = 1$ — “uma hora”.



Par ou ímpar

numero % 2 == 0 decide

o teste de paridade mais usado do semestre



Ciclos e alternância

índice % N percorre 0..N-1

relógios, listas circulares, alternar cores



Extrair informação

123 % 10 → 3

isola o último dígito de um número

Potência e raízes

python3 — ** (potência)

```
>>> 2 ** 10 → 1024
>>> 5 ** 2 → 25
>>> 9 ** 0.5 → 3.0 # raiz quadrada
>>> 27 ** (1 / 3) → 3.0 # raiz cúbica
>>> 2 ** -1 → 0.5 # expoente negativo inverte
```



Em Python, ^ NÃO é potência: é XOR bit a bit

```
>>> 2 ^ 3 → 1 # XOR, não potência
>>> 2 ** 3 → 8 # potência de verdade
```

Quem vem do Excel cai nessa.



Expoente flexível

Pode ser int, float ou negativo — 2^{-1} inverte o valor.



Raízes via expoente

Raiz quadrada = $** 0.5$; raiz N-ésima = $** (1/N)$.



Potências de 2 por todo lado

Bits e bytes crescem em potências de 2 — é a base da memória.

1024

= 2^{10} = 1 KiB

o “kilo” da computação

Operadores de atribuição

Partindo de `x = 10` em todas as linhas:

Operador	Exemplo	Equivale a	Resultado
<code>=</code>	<code>x = 10</code>	atribuição simples	10
<code>+=</code>	<code>x += 3</code>	<code>x = x + 3</code>	13
<code>-=</code>	<code>x -= 3</code>	<code>x = x - 3</code>	7
<code>*=</code>	<code>x *= 3</code>	<code>x = x * 3</code>	30
<code>/=</code>	<code>x /= 4</code>	<code>x = x / 4</code>	2.5
<code>//=</code>	<code>x //= 3</code>	<code>x = x // 3</code>	3
<code>%=</code>	<code>x %= 3</code>	<code>x = x % 3</code>	1
<code>**=</code>	<code>x **= 2</code>	<code>x = x ** 2</code>	100



Todo operador aritmético tem sua versão composta — leia `x += 3` como “x recebe ele mesmo mais 3”.

Acumuladores na prática



acumulador.py

```
soma = 0      # ponto de partida: neutro
soma += 4     # agora vale 4
soma += 3     # agora vale 7
soma += 10    # agora vale 17
print(soma)   # 17
```

EVOLUÇÃO DE soma



- Acumular é atualizar o mesmo valor ao longo do programa — um nome, muitos estados.
- Contador é o mesmo padrão com passo 1: contador += 1.
- Comece com 0 (neutro da soma) — ou com 1, se for produto (total *= fator).



Sem a linha `soma = 0`, o `+=` falha: **NameError** — a variável precisa existir antes de ser acumulada.

Somar 3 notas e dividir por 3 no final é exatamente esse padrão + uma divisão.

Precedência dos operadores

Nível	Operadores	O que faz
1º	()	parênteses — avaliam primeiro
2º	**	potência
3º	+X -X	sinal unário
4º	* / // %	multiplicativos
5º	+ -	aditivos
6º	= += -= ...	atribuições — por último

DESEMPATE

Mesmo nível? Resolva da esquerda para a direita.

$2 ** 3 ** 2 = 2 ** (3 ** 2) = 512$

exceção: ** agrupa da direita para a esquerda

SEM PARÊNTESES

$2 + 3 * 4$

→ **14** *a multiplicação vence*

COM PARÊNTESES

$(2 + 3) * 4$

→ **20** *os parênteses vencem*



Ninguém decora tabelas — **parênteses vencem sempre.**

Resolvendo passo a passo

AVALIANDO COM AS REGRAS

$$2 + 3 * 4 ** 2 - 10 // 5$$

1 Potência

$$4 ** 2 \rightarrow 16$$

$$\text{fica: } 2 + 3 * 16 - 10 // 5$$

2 Multiplicação e divisão

$$3 * 16 \rightarrow 48 \quad \cdot \quad 10 // 5 \rightarrow 2$$

$$\text{fica: } 2 + 48 - 2$$

3 Adição e subtração (esq. → dir.)

$$2 + 48 \rightarrow 50$$

$$\text{fica: } 50 - 2$$

4 Resultado final

$$= 48$$

Parênteses e legibilidade

$$(2 + 3) * 4$$

→ **20**

parênteses mudam o rumo

$$2 + (3 * 4)$$

→ **14**

explícito, mesmo resultado

$$2 + 3 * 4$$

→ **14**

as regras fazem o serviço

- Parênteses têm a precedência máxima — vencem qualquer operador.
- Use-os para tornar a intenção explícita, mesmo quando não são obrigatórios.
- Código é lido muito mais vezes do que é escrito — clareza compensa.

CASO REAL — MÉDIA DE 3 NOTAS

media = (n1 + n2 + n3) / 3

Sem os parênteses, o Python calcularia **n1 + n2 + (n3 / 3)** — e a média sairia errada.

Regra prática da turma: “se precisei de 3 segundos para decifrar, ganha parênteses”.

Tabela resumo — vale foto



ARITMÉTICA

+	-	soma e subtração
*		multiplicação
/		divisão real — sempre float
//		divisão inteira — piso
%		resto da divisão
**		potência



ATRIBUIÇÃO

x = v	guarda v em x
x += v	equivale a $x = x + v$
família	<code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>//=</code> <code>%=</code> <code>**=</code>
acumulador	nasce com 0 (soma) ou 1 (produto)



PRECEDÊNCIA

1°	()
2°	**
3°	* / // %
4°	+ -
5°	= += ...

desempate: esquerda → direita

Constantes em MAIÚSCULAS

nomes em snake_case

type() revela o tipo

/ devolve float

Erros comuns de iniciante



= não é ==

```
x = 5    # guarda 5 em x
x == 5   # pergunta: x é igual a 5?
```

= atribui um valor; == compara dois valores. Confundir os dois é o erro clássico — == é tema do próximo módulo.



/ quando queria //

```
5 / 2 → 2.5      5 // 2 → 2
```

A barra simples traz decimais (float). Faça uma escolha consciente conforme o problema pedir.



Divisão por zero

```
10 / 0 → ZeroDivisionError
```

O programa interrompe na hora. Verifique o divisor antes de dividir — tratamento elegante vem com try/except.



Misturar tipos

```
"2" + 3 → TypeError
```

Converta antes: int("2") + 3 → 5. E leia a mensagem do traceback com calma — ela diz exatamente o que aconteceu.

Prática: o que imprime?

DESAFIO 1

```
x = 5
x = x + 3
x *= 2
print(x)
```

acompanhe o valor de x linha a linha

→ 16

DESAFIO 2

```
print(17 // 5, 17 % 5)
```

duas saídas, separadas por espaço

→ 3 2

DESAFIO 3

```
print(2 + 3 * 2 ** 2)
```

aplique a precedência antes de somar

→ 14



Dinâmica — 2 minutos em duplas: resolva mentalmente antes de revelar; depois confira digitando no interpretador.

Para levar do Módulo 2



Variáveis nomeiam valores guardados na memória



Constantes: convenção em MAIÚSCULAS



Expressões avaliam para um único valor



7 operadores aritméticos: + - * / // % **



Atribuição composta compacta o acumulador



Precedência: () → ** → * / // % → + - → =



Pratique agora: abra o terminal, digite **python3** e refaça cada exemplo do módulo — errar no terminal é grátis.